Python desde cero

Variables, control de flujo, colecciones, funciones, ficheros y clases

CódigoClaro · Material didáctico de apoyo · Edición octubre de 2026

Cómo estudiar este material

Lee el concepto, escribe los ejemplos sin copiar y modifica los casos de prueba. Los fragmentos son ejemplos didácticos: adapta nombres, tipos y validaciones al enunciado real.

1. Variables y tipos

Python es un lenguaje dinámico. La indentación define bloques. Los tipos comunes incluyen int, float, str, bool, list, tuple, set y dict.

```
nombre = "Luz"
edad = 21
modulos = ["Programación", "Bases de datos"]
print(f"{nombre} cursa {len(modulos)} módulos")
```

2. Condiciones y bucles

```
nota = 7
if nota >= 5:
    print("Aprobado")
else:
    print("Recuperación")

for numero in range(3):
    print(numero)

contador = 0
while contador < 3:
    contador += 1
```

range(3) genera 0, 1 y 2. Usa for para iterar colecciones y while cuando la repetición depende de una condición. Comprueba que los bucles while puedan terminar.

3. Colecciones

```
notas = [6, 8, 9]
media = sum(notas) / len(notas)

alumno = {"nombre": "Luz", "nota": media}
for clave, valor in alumno.items():
    print(clave, valor)
```

List es mutable y ordenada; tuple es inmutable; set guarda elementos únicos; dict relaciona claves con valores. Comprueba que la lista no esté vacía antes de dividir por su longitud.

4. Funciones y módulos

```
def calcular_iva(precio, tipo=0.21):
    if precio < 0:
        raise ValueError("Precio negativo")
    return precio * (1 + tipo)

print(calcular_iva(100))
```

Las funciones ayudan a reutilizar lógica. Usa nombres descriptivos, parámetros explícitos y docstrings cuando la función lo necesite. Evita valores mutables como valores predeterminados sin entender su comportamiento.

5. Excepciones y ficheros

```
try:
    edad = int(input("Edad: "))
except ValueError:
    print("Introduce un entero válido")

with open("datos.txt", "w", encoding="utf-8") as archivo:
    archivo.write("Hola\n")
```

try/except permite gestionar errores esperados. with cierra el fichero automáticamente. No captures Exception de forma indiscriminada si no puedes recuperarte del error.

6. Clases

```
class Producto:
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio

    def precio_con_iva(self, tipo=0.21):
        return self.precio * (1 + tipo)

p = Producto("Libro", 20)
print(p.precio_con_iva())
```

7. Ejercicios

• Crea una función que devuelva la palabra más larga de una lista.

• Cuenta las vocales de una frase sin distinguir mayúsculas.

• Lee un CSV y calcula la media de una columna, gestionando datos inválidos.

• Modela una clase CuentaBancaria con ingresar y retirar validando importes.

Buenas prácticas

• Sigue PEP 8 y usa nombres descriptivos.

• Divide el programa en funciones pequeñas.

• Añade casos límite: lista vacía, texto vacío, número negativo y fichero inexistente.